Multimedia Computing, Assignment 1: Solution

Conventions: Python only; audio with numpy, matplotlib and scipy.io; image with numpy and OpenCV; video with OpenCV only, as the assignment requires. Grayscale uses `cv2.cvtColor(..., cv2.COLOR_BGR2GRAY)`. 8-bit results are cast with `np.uint8` the same way the professor's gamma code does (L6 p.47).

Each part has a **Source** line with the slide pages the method comes from (L6 p.47 = Lecture 6 slides, PDF page 47; plain text) and a **Notes** link that opens the md notes section explaining it.

Due: 2026-10-25 23:59 (eClass shows 23:55). Late: up to 3 h costs 30%, 3 to 6 h costs 50%, up to 24 h costs 70%, after that 0.
Upload to eClass (7 loose files, no zip, no folders): `1_audio_waveform.py`, `2_audio_compression.py`, `1_image_log_transform.py`, `2_image_sobel_edges.py`, `1_video_downscale.py`, `2_video_blur.py`, `requirements.txt`.
Run: put the 3 `original_*` files next to the scripts and run each with `python <script>.py`. The outputs (png, wav, mp4) are generated, not submitted.

Audio

Task 1: Waveform Analysis (2 pts)

> Use the provided original_audio.wav. Extract the waveform and plot it using Python libraries (e.g., matplotlib).
> Save the plot as 1_audio_waveform.png. Figure size must be (10, 4) inches using matplotlib.pyplot.
> Submit 1_audio_waveform.py. Only NumPy, Matplotlib and SciPy (scipy.io) are allowed (librosa, wave, etc. give 0).

Source: L2 p.19-21 sampling $x_s(n) = x(nT)$, $T = 1/f$ (time axis $t = n/f_s$) · L8 p.28 plotting
Notes: Lecture 2 › Sampling · Lecture 8 › Plotting

Answer: `1_audio_waveform.py`

```python
import numpy as np
import matplotlib
matplotlib.use("Agg")  # save to file, no window
import matplotlib.pyplot as plt
from scipy.io import wavfile

INPUT = "original_audio.wav"
OUTPUT = "1_audio_waveform.png"

sample_rate, data = wavfile.read(INPUT)
time = np.arange(data.shape[0]) / sample_rate  # seconds

plt.figure(figsize=(10, 4))
if data.ndim == 1:
    plt.plot(time, data, linewidth=0.5)
else:  # stereo: one line per channel
    for ch in range(data.shape[1]):
        plt.plot(time, data[:, ch], linewidth=0.5, label=f"Channel {ch + 1}")
    plt.legend(loc="upper right")
plt.title("Waveform of original_audio.wav")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.savefig(OUTPUT)
plt.close()
print(f"sample rate = {sample_rate} Hz, samples = {data.shape[0]}, saved {OUTPUT}")
```

How:

Item	Value
File	16-bit PCM, stereo, 44100 Hz, 533,342 samples per channel
Duration	533342 / 44100 = 12.094 s
Time axis	$t = n/f_s$ (sample index divided by sample rate)
Plot	both channels, figure (10, 4) in, saved at 100 dpi = 1000 x 400 px

Task 2: Audio Compression (2 pts)

> Compress original_audio.wav: apply a suitable compression technique and reduce the sample rate twice (by a factor of 2).
> Requirement: compression_ratio = compressed_size / original_size < 0.7.
> Save as 2_audio_compression.wav. Submit 2_audio_compression.py. Same library rule as Task 1.

Source: L2 p.34-35 bit rate = sampling rate x bits x channels · L2 p.47-51 Nyquist and aliasing · L3 p.11-12 data rate table (44.1 / 22.05 / 11.025 kHz)
Notes: Lecture 2 › Halving the sample rate · Lecture 2 › Bit rate

Answer: `2_audio_compression.py`, **compression_ratio = 0.500** (< 0.7)

```python
import os
import numpy as np
from scipy.io import wavfile

INPUT = "original_audio.wav"
OUTPUT = "2_audio_compression.wav"

sample_rate, data = wavfile.read(INPUT)

# Reduce the sample rate by a factor of 2 (downsampling: keep every 2nd sample)
new_rate = sample_rate // 2
compressed = data[::2].astype(np.int16)  # stored as 16-bit PCM

wavfile.write(OUTPUT, new_rate, compressed)

original_size = os.path.getsize(INPUT)
compressed_size = os.path.getsize(OUTPUT)
compression_ratio = compressed_size / original_size
print(f"sample rate: {sample_rate} Hz -> {new_rate} Hz")
print(f"size: {original_size} -> {compressed_size} bytes")
print(f"compression_ratio = {compression_ratio:.3f} (must be < 0.7)")
```

How:

Item	Original	Compressed
Sample rate	44100 Hz	22050 Hz (keep every 2nd sample, `data[::2]`)
Samples per channel	533,342	266,671
Format	16-bit stereo	16-bit stereo
Bit rate	44100 x 16 x 2 = 1,411,200 bit/s	705,600 bit/s
File size	2,133,688 B	1,066,728 B
Ratio		1066728 / 2133688 = **0.49995**

Halving the rate halves the highest representable frequency (Nyquist) from 22.05 kHz to 11.025 kHz. Content above that can alias, but voice-band audio like this file is barely affected.

Image

Task 1: Logarithmic Intensity Transform (2 pts)

> Read original_image.png (RGB) and convert it to grayscale. Normalize it first.
> Apply $I_{new} = c \cdot \log\left(1 + I_{old}\right)$ with $c = 255 / \log\left(1 + \max\left(I_{old}\right)\right)$.
> Save as 1_image_log_transform.png. Submit 1_image_log_transform.py. Only Pillow, NumPy and OpenCV are allowed.

Source: log transform is not on the slides (only named on L9 p.2); method pattern from L6 p.44-45 intensity transform = point process, L6 p.47 gamma code `np.array(255*(img/255)**gamma, dtype='uint8')` (normalize, transform, back to uint8) · L9 p.13 $s = T(r)$
Notes: Lecture 6 › Intensity transformations and gamma · Lecture 9 › Intensity transformations (log, textbook context)

Answer: `1_image_log_transform.py`, **c = 255 / ln 2 = 367.887**, output range 75..255

```python
import numpy as np
import cv2

INPUT = "original_image.png"
OUTPUT = "1_image_log_transform.png"

# 1) Read the RGB image (OpenCV loads it in BGR order) and convert to grayscale
img = cv2.imread(INPUT, cv2.IMREAD_COLOR)
if img is None:
    raise SystemExit(f"Cannot read {INPUT}")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 2) Normalize intensities to [0, 1]
I_old = gray.astype(np.float64) / 255.0

# 3) Log transform: I_new = c * log(1 + I_old),  c = 255 / log(1 + max(I_old))
c = 255.0 / np.log(1.0 + np.max(I_old))
I_new = c * np.log(1.0 + I_old)

# 4) Back to 8-bit (uint8 cast, as in the lecture's gamma example) and save
result = np.clip(I_new, 0, 255).astype(np.uint8)
cv2.imwrite(OUTPUT, result)
print(f"c = {c:.4f}, output range {result.min()}..{result.max()}, saved {OUTPUT}")
```

How:

Step	Value
Image	512 x 512 palette PNG; `cv2.imread` gives BGR, then `COLOR_BGR2GRAY`
Gray range	58..255, normalized I_{old} = gray / 255, so $\max\ (\ I_{old}\ ) = 1.0$
c	255 / ln(1 + 1) = **367.887**
Check	gray 0 -> 0, 58 -> 75.39 (stored 75), 128 -> 149.65 (149), 255 -> 255
Effect	dark tones are expanded, bright ones compressed; mean 213.3 -> 220.3

The log base doesn't matter: c uses the same log, so it cancels.

Task 2: Sobel Edge Detection (3 pts)

> Read original_image.png and convert it to grayscale.
> Apply Sobel in x and y with $G_x = [\ [\ -1, 0, 1\ ],\ [\ -2, 0, 2\ ],\ [\ -1, 0, 1\ ]\ ]$ and $G_y = [\ [\ -1, -2, -1\ ],\ [\ 0, 0, 0\ ],\ [\ 1, 2, 1\ ]\ ]$.
> Combine: $G = \sqrt{G_x^2 + G_y^2}$. Save as 2_image_sobel_edges.png. Submit 2_image_sobel_edges.py.

Source: L11 p.43 Sobel M_x, M_y (same kernels as the assignment) · L11 p.37 magnitude $\sqrt{H_r^2 + H_c^2}$ or $|H_r| + |H_c|$, direction $\tan^{-1}\ (\ H_c / H_r\ )$ · L11 p.14 applying a mask (no flip)
Notes: Lecture 11 › First order derivative operators · Lecture 11 › Convolution and borders

Answer: `2_image_sobel_edges.py`

```
        import numpy as np
        import cv2

        INPUT = "original_image.png"
        OUTPUT = "2_image_sobel_edges.png"

        # 1) Read the RGB image (BGR in OpenCV) and convert to grayscale
        img = cv2.imread(INPUT, cv2.IMREAD_COLOR)
        if img is None:
            raise SystemExit(f"Cannot read {INPUT}")
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float64)

        # 2) Sobel kernels exactly as given in the assignment
        Gx_kernel = np.array([[-1, 0, 1],
                              [-2, 0, 2],
                              [-1, 0, 1]], dtype=np.float64)
        Gy_kernel = np.array([[-1, -2, -1],
                              [ 0,  0,  0],
                              [ 1,  2,  1]], dtype=np.float64)

        Gx = cv2.filter2D(gray, cv2.CV_64F, Gx_kernel)
        Gy = cv2.filter2D(gray, cv2.CV_64F, Gy_kernel)

        # 3) Gradient magnitude G = sqrt(Gx^2 + Gy^2)
        G = np.sqrt(Gx ** 2 + Gy ** 2)

        # 4) Keep the values in the valid 8-bit range (0..255) and save as PNG
        result = np.clip(G, 0, 255).astype(np.uint8)
        cv2.imwrite(OUTPUT, result)
        print(f"gradient range {G.min():.1f}..{G.max():.1f} (clipped to 0..255), saved
{OUTPUT}")
```

How:

Step	Value
Filtering	`cv2.filter2D` with the exact kernels (identical to `cv2.Sobel(ksize=3)`, checked)
Magnitude	$G = \sqrt{G_x^2 + G_y^2}$, range 0..767.2
To 8-bit	clip to 0..255 (0.39% of pixels saturate). This is also what `cv2.imwrite` does with a float image
Mini check	patch rows `[10 10 200]` x3: $G_x = 760$, $G_y = 0$, $G = 760$ (vertical edge)

Sign or orientation of the kernels doesn't change G (squares), so correlation (`filter2D`) vs convolution gives the same result.

Video

Task 1: Downscaling by 50% (3 pts)

> Input original_video.mp4. Resize each frame to 50% width and height (original dimensions divided by 2) with `cv2.resize(..., interpolation=cv2.INTER_LINEAR)` only.
> Keep the original fps. Output 2_video_downscale.mp4 (the submission list calls it

> 1_video_downscale.mp4). Submit 1_video_downscale.py.
> Graded: width, height and fps vs the reference.

Source: L6 p.36 resampling: nearest / bilinear / bicubic · L8 p.32 imresize
Notes: Lecture 6 › Image resampling and interpolation

Answer: `1_video_downscale.py`, **320x240 -> 160x120, 15 fps, 205 frames**

```
        import cv2

        INPUT = "original_video.mp4"
        OUTPUT = "1_video_downscale.mp4"
        # The task text also names the output "2_video_downscale.mp4"; write both so either
name is found.
        OUTPUT_ALT = "2_video_downscale.mp4"

        cap = cv2.VideoCapture(INPUT)
        if not cap.isOpened():
            raise SystemExit(f"Cannot open {INPUT}")

        fps = cap.get(cv2.CAP_PROP_FPS)
        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

        # 50% of the original size
        new_width = width // 2
        new_height = height // 2

        fourcc = cv2.VideoWriter_fourcc(*"mp4v")
        writers = [cv2.VideoWriter(name, fourcc, fps, (new_width, new_height))
                   for name in (OUTPUT, OUTPUT_ALT)]

        while True:
            ret, frame = cap.read()
            if not ret:
                break
            small = cv2.resize(frame, (new_width, new_height),
interpolation=cv2.INTER_LINEAR)
            for w in writers:
                w.write(small)

        cap.release()
        for w in writers:
            w.release()
        print(f"{width}x{height} -> {new_width}x{new_height} at {fps} fps, saved {OUTPUT}")
```

How:

Item	Original	Output
Size	320 x 240	320//2 x 240//2 = **160 x 120**
FPS	15	15 (same)
Frames	205	205
Codec	h264	mp4v (MPEG-4 Part 2, written by OpenCV)

The assignment names the output both `2_video_downscale.mp4` and `1_video_downscale.mp4`. The script writes both, so the grader finds either.

Task 2: Video Blurring (3 pts)

> Input original_video.mp4. Apply `cv2.GaussianBlur(frame, (15, 15), 0)` to every frame. Keep fps, width and height. Output 2_video_blur.mp4. Submit 2_video_blur.py.

Source: L11 p.19 Gaussian smoothing $G = e^{-\left(x^2+y^2\right)/2\sigma^2}$ and the 3x3 mask (1/16)[1 2 1; 2 4 2; 1 2 1] · L11 p.16-18 averaging masks
Notes: Lecture 11 › Blurring: average and Gaussian masks

Answer: `2_video_blur.py`, **320x240, 15 fps, 205 frames**

```
        import cv2

        INPUT = "original_video.mp4"
        OUTPUT = "2_video_blur.mp4"

        cap = cv2.VideoCapture(INPUT)
        if not cap.isOpened():
            raise SystemExit(f"Cannot open {INPUT}")

        fps = cap.get(cv2.CAP_PROP_FPS)
        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

        fourcc = cv2.VideoWriter_fourcc(*"mp4v")
        out = cv2.VideoWriter(OUTPUT, fourcc, fps, (width, height))

        while True:
            ret, frame = cap.read()
            if not ret:
                break
            # Gaussian blur: 15x15 kernel, sigma = 0 (OpenCV derives sigma from the kernel
size)
            blurred = cv2.GaussianBlur(frame, (15, 15), 0)
            out.write(blurred)

        cap.release()
        out.release()
        print(f"{width}x{height} at {fps} fps, saved {OUTPUT}")
```

How:

Item	Value
Kernel	15 x 15 Gaussian; sigma = 0 means OpenCV derives it: $\sigma = 0.3\left(\ \left(k-1\right)/2-1\ \right)+0.8 = 2.6$
Output	same 320 x 240, 15 fps, 205 frames

requirements.txt

```
numpy>=1.24
scipy>=1.10
matplotlib>=3.7
opencv-python>=4.8
```

Answer summary

Task	File	Key result
Audio 1	1_audio_waveform.py	1000 x 400 px plot (10 x 4 in), 12.094 s, stereo
Audio 2	2_audio_compression.py	44100 -> 22050 Hz, ratio **0.500**
Image 1	1_image_log_transform.py	c = 255/ln 2 = **367.887**, range 75..255
Image 2	2_image_sobel_edges.py	$G = \sqrt{G_x^2 + G_y^2}$, max 767.2, clipped to 255
Video 1	1_video_downscale.py	320x240 -> **160x120**, 15 fps
Video 2	2_video_blur.py	GaussianBlur (15,15), 0; 320x240, 15 fps

Judgment calls (the assignment doesn't fix them): stereo plotted as 2 lines; Sobel output clipped, not rescaled to max; log result cast with `uint8` (truncation). Each changes at most a few gray levels or the plot styling.